Savitribai Phule Pune University

S. Y. B. C. A. (Science) 2024 Pattern (NEP-2020)

Semester-II

Lab Course on

ELS – 292 - MNP: Communication Networks

Lab Book for
ELS – 292 - MNP: Communication Networks Laboratory

Name:_____

College Name: K.T.H.M College, Nashik

Roll No. : _____ Division:_____

Academic Year: 2025-26

Introduction

1. About the Workbook:

This workbook is intended to be used by SYBCA (Science) students for the Communication Networks Laboratory Assignments in Semester–II. This workbook is designed by considering all the practical concepts / topics mentioned in syllabus.

2. The Objectives of this Workbook are:

- Defining the scope of the course.
- To bring the uniformity in the practical conduction and implementation in all colleges affiliated to SPPU.
- To have continuous assessment of the course and students.
- Providing ready reference for the students during practical implementation.
- Provide more options to students so that they can have good practice before facing the examination.
- Catering to the demand of slow and fast learners and accordingly providing the practice assignments to them.

3. How to use this Workbook:

Communication Networks Laboratory workbook is divided into thirteen assignments. The assignments comprise of activities to be carried out while doing each practical. The students have to study the respective practical kit, make connections, insert appropriate records or observations and then perform the activities specified in each of the assignments.

4. Instructions to the students:

Please read the following instructions carefully and follow them.

- Students are expected to carry this workbook every time they come to the lab for practical.
- Students should prepare for the assignment by reading the relevant material which is mentioned in ready reference.
- Instructor will specify which problems to solve in the lab during the allotted slot and student should complete them and get verified by the instructor. However, student should spend additional hours in Lab and at home to cover all workbook assignments if needed.
- Students will be assessed for each assignment on a scale from 0 to 5

Not done	0
Incomplete	1
Late Complete	2
Needs improvement	3
Complete	4
Well Done	5

5. Instruction to the Instructors:

- Make sure that students should follow above instructions.
- Explain the practical assignment and related concepts using white board if required or by demonstrating on the circuit board.
- Evaluate each assignment carried out by a student on a scale of 5 as specified above by ticking appropriate box.
- The value should also be entered on assignment completion page of the respective Lab course.

Table of Contents

Assignment Completion Sheet	Name of Practical	Page No.	Remark
Assignment No. 1	To study Configuration of IP and MAC address.	1	
Assignment No. 3	To convert the given IP addresses from dotted-decimal notion to binary notation.	5	
Assignment No. 4	To convert the given IP addresses from binary to dotted decimal notion.	8	
Assignment No. 5	To find NetID and Host ID of the given IP addresses.	11	
Assignment No. 6	To find class of the given IP addresses.	15	
Assignment No. 7	To study Arduino based LED pattern generation.	23	
Assignment No. 8	To study of Bluetooth.	27	
Assignment No. 9	To study the RFID.	33	
Assignment No. 10	Point to point communication using Zig-Bee.	37	
Assignment No. 11	Wireless sensor node used for street light control.	41	
Assignment No. 12	Wireless sensor node used for environment monitoring.	43	
Assignment No. 13	Wireless sensor node used for security system.	47	

CERTIFICATE

This is to certify that Mr./Miss. of _____S.Y.B.C.A. (Science) Semester-II, Roll No.___Exam Seat No. _____has satisfactorily completed ___out of 13 experiments of Practical course Communication Networks Laboratory during academic year 2025-26 and same have examined and signed by In charge.

Practical In-Charge **Internal Examiner** **External Examiner**

S.Y.B.C.A. (Science) Semester-IV (NEP-2024 Pattern)
Lab Experiment No.-1
Configuration of IP and MAC Address

Aim: To Study Configuration of IP and MAC Address.

Objectives:

1. To study the concept and structure of IP and MAC addresses.

2. To identify the IP and MAC address of a system.

3. To configure a static IP address manually.

4. To verify connectivity using network utilities.

Apparatus/Software Required:

1. A computer system with Windows/Linux OS

2. Network Interface Card (NIC)

3. Command Prompt/Terminal

4. Ethernet or Wi-Fi connection

IP Address

- An IP address (Internet Protocol Address) is a logical address assigned to a device connected to a network. It is used to identify and communicate with devices over a network. IP addresses can be static or dynamic.

- Used for identifying and locating devices on a network.

- Can be changed by the user or network administrator.

- IPv4: 32-bit address, represented as four decimal octets (e.g., 192.168.1.10)

- IPv6: 128-bit address, represented in hexadecimal (e.g., 2001:0db8:85a3::8a2e:0370:7334)

Types of IP Address

- Static IP-Manually configured/ assigned and fixed.
- Dynamic IP-Assigned automatically using DHCP (Dynamic Host Configuration Protocol).

MAC Address

- A MAC address (Media Access Control Address) is a physical address assigned to the Network Interface Card by the manufacturer. It is unique for each device and is used for communication within the local network.
- It is a 48-bit address represented in hexadecimal format

  (e.g., 00-1A-92-3F-B6-3C).
- IP address operates at the Network Layer, while MAC address works at the Data Link Layer of the OSI model.

Assignment-1: Viewing IP and MAC Address of your PC

Follow the following steps on your PC:

For Windows OS

1. Open Command Prompt

2. Type following command and press enter:

"ipconfig /all"

For Linux

1. Open Terminal.

2. Type the command:

ifconfig

or

ip a

3. Note down the IPv4 Address, Subnet Mask, Default Gateway, and Physical (MAC) Address.

IPv4 Address:	
Subnet Mask:	
Default Gateway:	
Physical (MAC) Address:	

Assignment-2: Configuring IP Address (Windows)

Steps:

1. Open Control Panel

2. Go to Network and Sharing Center

3. Click Change adapter settings

4. Right-click on network Properties

5. Select Internet Protocol Version 4 (IPv4)

6. Click Properties

7. Choose:

a. Obtain IP automatically (DHCP), or

b. Use the following IP address and enter

IP Address: 192.168.1.10

Subnet Mask: 255.255.255.0

Default Gateway: 192.168.1.1

DNS: 8.8.8.8

8. Click OK

Assignment-3: Verify Configuration

- To check connectivity:
  ping 192.168.1.1

- To confirm configuration.
  ipconfig /all (Windows)
  ipconfig (Linux)

Result: The system's IP and MAC addresses were successfully identified and configured. Network connectivity was verified using the ping command.

Conclusion: The experiment demonstrated how to view and configure both logical (IP) and physical (MAC) addresses in a networked computer system. Understanding this configuration is essential for establishing and troubleshooting network communication

Questions:

1. What is difference between Logical address and physical address?
2. State the use of ping command.
3. What is the difference between IP and MAC address?
4. At which OSI layers do IP and MAC operate?
5. What is DHCP and its purpose?
6. What command displays network configuration details?
7. Can two devices have the same MAC address?

Signature of Instructor

S.Y.B.C.A. (Science) Semester-IV (NEP-2024 Pattern)
Lab Experiment No.-3
IP Address Conversion: Dotted Decimal to Binary Notation

Aim: To convert the given IP addresses from dotted-decimal notion to binary notation.

Objectives:

1. To understand the structure of IPv4 addressing.

2. To learn binary conversion of decimal octets in an IP address.

3. To verify correct representation of IP addresses in binary format.

Apparatus/Software Required:

1. A computer system (Windows/Linux)

2. Calculator/Scientific calculator

3. Pen and paper for manual conversion

4. (Optional) IP conversion software or online tool for verification

IPv4 Header Format:

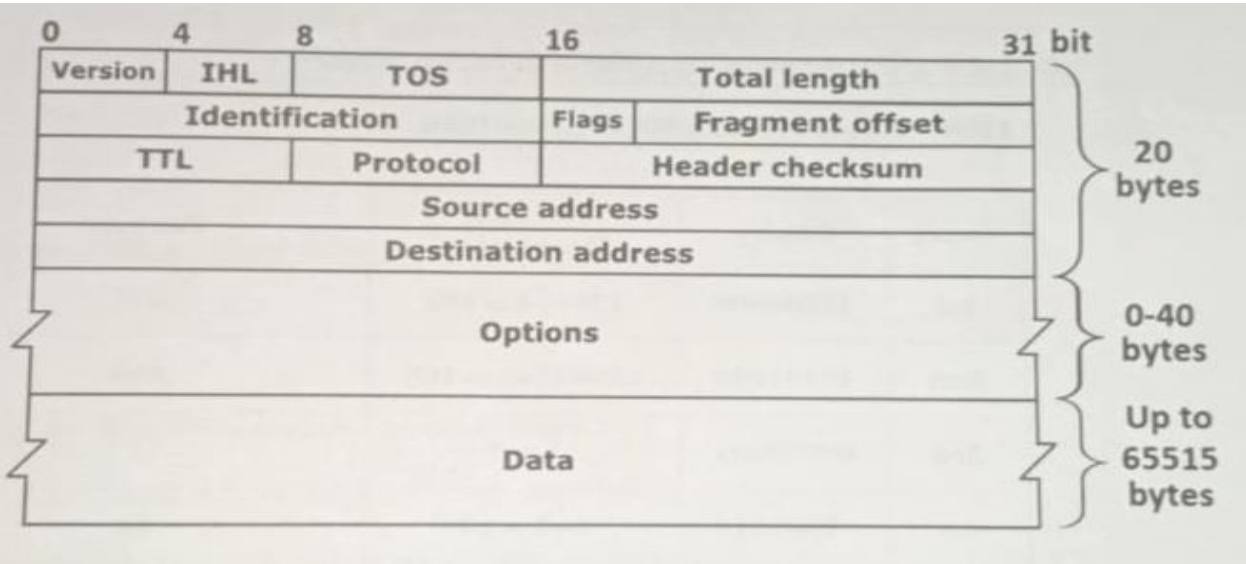


Figure1: The IPv4 header

1. The IPv4 Address:

- An IPv4 address is a 32-bit address used to identify a device in a network
- It is represented in dotted-decimal notation four octets separated by dots,

  eg. 192.168.1.10
- Each octet (byte) can take a value between 0 and 255.
- Every octet corresponds to 8 bits, giving a total of 32 bits for the entire address.

2. Binary Conversion

- To convert a decimal number to binary Divide the number by 2 repeatedly and record remainders in reverse order.
- Example: Decimal 192 = Binary 11000000
- Let's apply for given IP Address: 192.168.1.10

Octet	Decimal	Binary Conversion	Binary (8 bits)
Ist	**192**	**128+64-192**	**11000000**
2nd	**168**	**128+32+8 168**	**10101000**
3rd	**1**	**1=1**	**00000001**
4th	**10**	**8+2=10**	**00001010**

- **Binary Notation: 11000000.10101000.00000001.00001010**

Assignment-1:

a. Find the IPv4 address of your PC and convert it into binary notation.

b. Note down in following table.

IPv4 address in Decimal Notation	
IPv4 address in Binary Notation	

Assignment-2:

a. Convert the given IPv4 address into binary notation

b. Also identify the wrong IPv4 address if any.

Sr. No.	Given IP Address	Binary notation
1	**148.162.1.12**	
2	**258.2.1.22**	
3	**127.256.0.1**	
4	**12.11.22.2**	
5	**192.168.1.10**	

Result: Thus, Studied IP Address Conversion: Dotted Decimal to Binary Notation.

Questions:

1. How many bits are there in an IPv4 address?
2. Why is the IP address divided into four octets?
3. What is the range of values for each octet in an IPv4 address?
4. Convert **148.162.1.12** into binary.
5. Differentiate between IPv4 and IPv6 addressing.
6. What is the range of value for each octet in an IPv4 address?
7. Write the total address space of IPv4 address. 5

Signature of Instructor

S.Y.B.C.A. (Science) Semester-IV (NEP-2024 Pattern)
Lab Experiment No.-4
IP Address Conversion: Binary Notation to Dotted Decimal

Aim: To convert the given IP addresses from binary to dotted decimal notion.

Objectives:

1. To understand the structure of IPv4 addressing and binary representation.

2. To perform binary-to-decimal conversion for each octet of an IPv4 address.

3. To verify correctness of the converted IP address.

Apparatus/Software Required:

1. A computer system (Windows/Linux)

2. Calculator/Scientific calculator

3. Pen and paper for manual conversion

4. (Optional) IP conversion software

IPv4 Header Format:

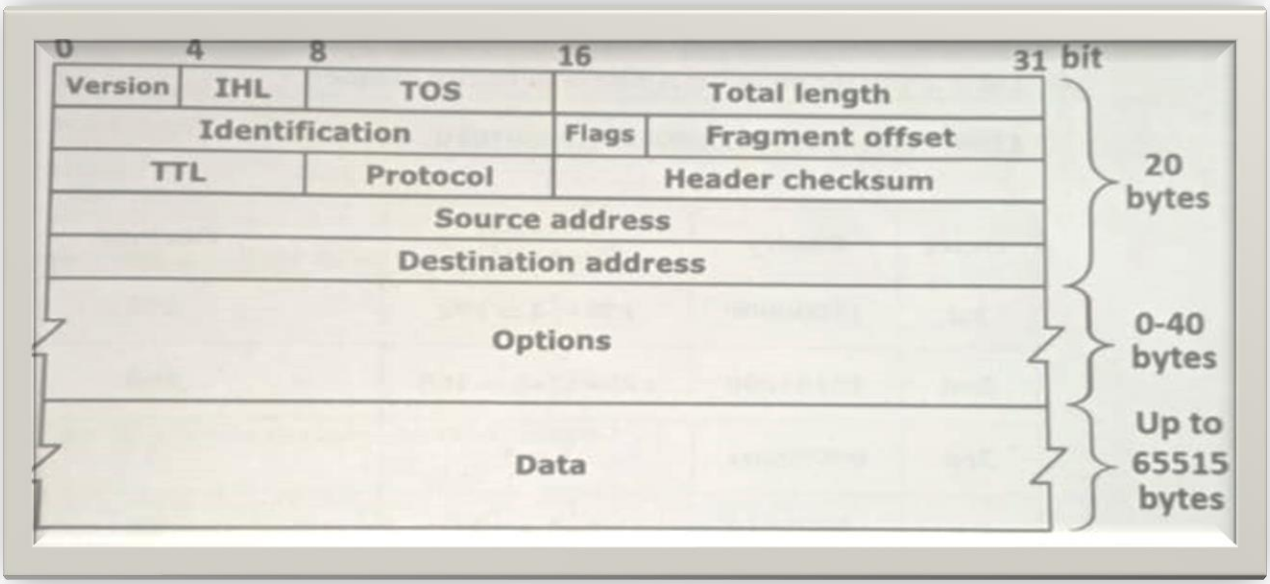


Figure1: The IPv4 header

1. The IPv4 Address:

- An IPv4 address is a 32-bit address used to identify a device in a network
- It is represented in dotted-decimal notation four octets separated by dots,

  eg. 192.168.1.10
- Each octet (byte) can take a value between 0 and 255.
- Every octet corresponds to 8 bits, giving a total of 32 bits for the entire address.

2. Binary to Decimal Conversion

- Each binary digit (bit) represents a power of 2, starting from the right (least significant bit)
- To convert binary to decimal, multiply each bit by its positional value and sum the results.

Bit Position	7	6	5	4	3	2	1	0
Weight (2^n)	128	64	32	16	8	4	2	1

- **Example:** Binary: 11000000
  = (1x128)+(1x64)+(0x32)+(0x16)+(0x8)+(0x4)+(0x2)+(0x1)
  =192
- Let's apply for given IP Address in binary notation:

  11000000.10101000.00000001.00001010

Octet	Binary	Decimal Conversion	Decimal
1st	**11000000**	**128+64-192**	**192**
2nd	**10101000**	**128+32+8 168**	**168**
3rd	**00000001**	**1=1**	**1**
4th	**00001010**	**8+2=10**	**10**

- **Decimal Notation: 192.168.1.10**

Assignment-1:

a. Find the IPv4 address of your PC and convert it into decimal notation.

b. Note down in following table.

IPv4 address in Binary Notation	
IPv4 address in Decimal Notation	

Assignment-2:

a. Convert the given IPv4 address into decimal notation

b. Also identify the wrong IPv4 address if any.

Sr. No.	Given IP Address	Decimal notation
1	**11000000.10101000.00000001.00001010**	
2	**01111111.00000000.00000000.00000000**	
3	**11110000.11001100.01010101.10111100**	

Result: Thus, Studied IP Address Conversion: Binary Notation to Dotted Decimal.

Questions:

1. How many bits are there in an IPv4 address?
2. Write the total address space of IPv4 address.
3. Find and comment on the error if any in following IP address:
   a) 111.56.045.78 b) 11100010.23.14.55 c) 221.34.7.45.8 d) 45.75.301.14
4. Why is the IP address divided into four octets?
5. What is the range of values for each octet in an IPv4 address?
6. Convert 11000000.10101000.00000001.00001010 into decimal notation.
7. Differentiate between IPv4 and IPv6 addressing.

Signature of Instructor

S.Y.B.C.A. (Science) Semester-IV (NEP-2024 Pattern)
Lab Experiment No.-5
NetID and Host ID of the Given IP Address

Aim: To Find NetID and Host ID of the Given IP Addresses

Objectives:
1. To understand the concept of NetID and Host ID in IPv4 addressing
2. To perform bitwise operations (AND) between IP address and subnet mask to compute NetID
3. To extract Host ID by bitwise subtraction or using mask complement.

Apparatus/Software Required:
1. Computer with text editor or paper & pen
2. Calculator (optional)
3. Command prompt/terminal (optional for verification)
4. (Optional) IP/subnet calculator tool for verification

The IP address format in terms of Net-ID and Host-ID.

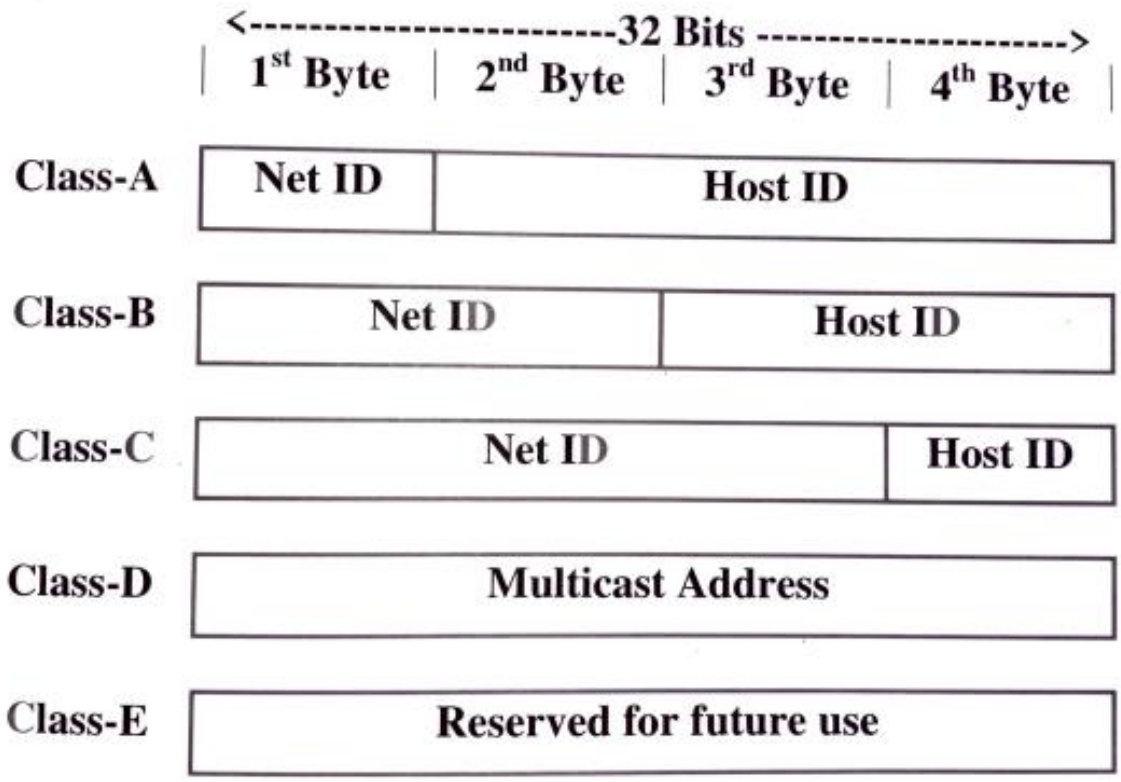


Figure: NetID and HostID

Assignment-1: Find the Net ID and Host ID of following IP address.

IP Address	Net ID	Host ID	Default subnet mask	Class
192.168.2.130/25				
198.0.13.21				
80.20.12.3				
134.23.120.8				

Assignment-2: Now Follow for your PC:

1. Start the PC
2. Check the IP address and not down
3. Find the Net ID and Host ID for given IP address
4. Find the network subnet mask.

IP Address	Net ID	Host ID	Default subnet mask

Theory:

1. An IPv4 address is a 32-bit number displayed as four decimal octets in dotted-decimal notation: a.b.c.d.

2. An IPv4 network is partitioned into NetID (network portion) and Host ID (host portion). The boundary is defined by the subnet mask (32-bit) or CIDR prefix in where n bits are network bits.

3. The NetID is obtained by performing a bitwise AND between the 32-bit IP address and the 32-bit subnet mask.

4. The Host ID can be obtained by:

a) bitwise AND between IP and the inverse mask (bitwise NOT of subnet mask), or
b) subtracting the NetID from the full IP (conceptually), or
c) writing the host bits (bits after the prefix) from IP.

5. Example: For IP 192.168.1.130/25 mask/25= 255.255.255.128.

NetID = 192.168.1.128. Host ID = 0.0.0.2 (or host number 2).

Procedure (Step-by-step)

1. Write the IP address and subnet mask in dotted-decimal. If mask given in CIDR/n. convert to dotted-decimal mask (see table below for common masks).

2. Convert each octet of the IP and mask to 8-bit binary. (Work octet-by-octet)

3. Perform bitwise AND for each corresponding pair of octets:

NetID_octet IP_octet AND Mask_octet

4. Write NetID by combining the resulting four NetID octets in dotted-decimal

5. Compute Host ID by:

a) Host octet IP_octet AND (NOT Mask octet) for each octet, or

b) Subtract NetID from IP (conceptually) and express result as host portion; or

d) Extract the lower (32-n) bits from IP (as binary) and convert to decimal per octet

6. Verify: NetID & Mask = NetID (trivial), and NetID ≤ IP ≤ Broadcast (depending on host bits).

7. Document results in observation table.

Example-1 (Classful/Class C default mask):

Given: IP = 192.168.10.45, Class C default mask 255.255.255.0 (/24).

Step 1- Binary conversion (octet by octet):

1st octet: 192 ---binary: 11000000

2nd octet: 168 ---binary: 10101000

3rd octet: 10---binary: 00001010

4th octet: 45--- binary: 00101101

Mask octets: 255---11111111,255---11111111, 255---11111111,0---00000000

Step 2- Bitwise AND (each octet):

Octet1: 11000000 AND 11111111=11000000 →decimal 192

Octet2: 10101000 AND 11111111=10101000 →decimal 168

Octet3: 00001010 AND 11111111=00001010 →decimal 10

Octet4: 00101101 AND 00000000=00000000→decimal 0

NetID: 192.168.10.0

Host ID (IP-NetID or IP AND NOT mask):

Octet-4 host bits: 00101101 AND 11111111-00101101 → decimal 45

So Host ID: 0.0.0.45 (or host number 45)

Result: Using bitwise AND between IP address and subout mask, the NetID and Host ID were successfully determined for the given IP addresses. The method applies for clausful defaults and classless CIDR masks equally.

Conclusion:

1. NetID is obtained by IP AND Mask
2. Host ID is obtained by IP AND (NOT Mask) or by subtracting NetID from IP.
3. CIDR masks allow flexible division of address space, the same bit wise method applies.
4. Accurate binary conversion and careful bitwise operations are essential to avoid mistakes.

Questions

1. Define NetID and Host ID
2. How do you compute NetID from an IP address and subnet mask? Show with an example.
3. Convert the IP 192.168.5.77 with mask /25 to NetID and Host ID.
4. What is the broadcast address for network 10.0.0.128/25
5. How many usable hosts are there in a /28 network?
6. Explain difference between classful and classless addressing.

Signature of Instructor

S.Y.B.C.A. (Science) Semester-IV (NEP-2024 Pattern)
Lab Experiment No.-6
Class of the Given IP Addresses

Aim: To Find Class of the Given IP Addresses

Objectives:

1. To understand the concept of classful addressing in IPv4.

2. To classify IP addresses according to their range and network usage

3. To identify default subnet masks and address ranges for each class.

Apparatus/Software Required:

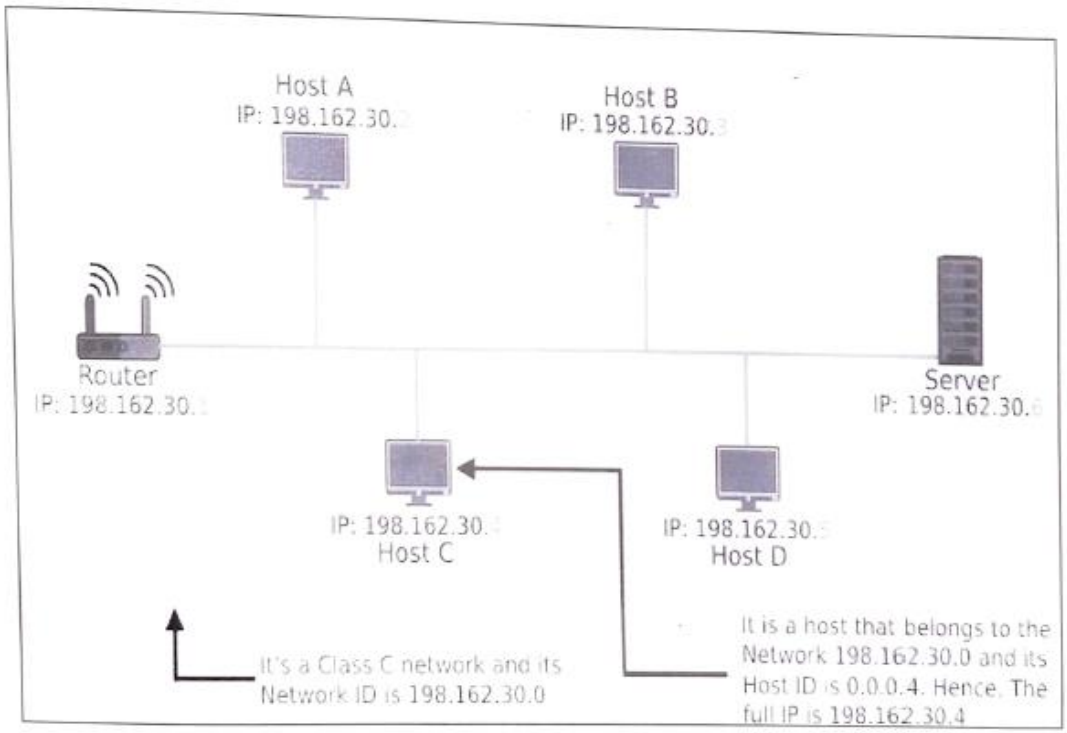
1. Computer system with operating system (Windows/Linux)

2. Text editor or pen-paper

3. Calculator (optional)

4. (Optional) IP classification simulation or command prompt

Diagram: A simple class C Network with IP address 198.162.30.0



Theory:

A] Need for Classful Addressing

The following points highlight the need for classful addressing.

1. IP Address Allocation becomes simplified

In classful addressing, the IPv4 address space is divided into five different classes (A, B, C.D and E), and each class has a fixed network ID and host ID.
This helps in allocation of addresses without any complex calculations.

Class A →very large networks

Class B → medium networks

Class C →small networks

2. Make Routing faster

The first few bits of an IP address represent the network portion and can be determined by the router. This helps reduce processing overhead and makes routing tables smaller.

Example:

- If address starts with 0-127, it's Class A
- 128-191→Class B
- 192-223→Class C

No need for subnet masks----routers inferred them automatically.

3. Reduce Administrative Complexity

Since subnet masks were fixed for each class, network administrators didn't need to design custom addressing schemes
This was helpful when networking knowledge and tools were limited.

4. Provide Structure to the Emerging Internet

In the early days, a structured system was needed so organizations could request

appropriate address blocks.
Classful addressing gave the Internet Assigned Numbers Authority (IANA) a standardized way to distribute IPs.

5. Supported different network sizes (Scalability)

With classes A. B, and C, organizations could get an address block that roughly matched their anticipated network size.
It prevented chaos in the IP allocation process.

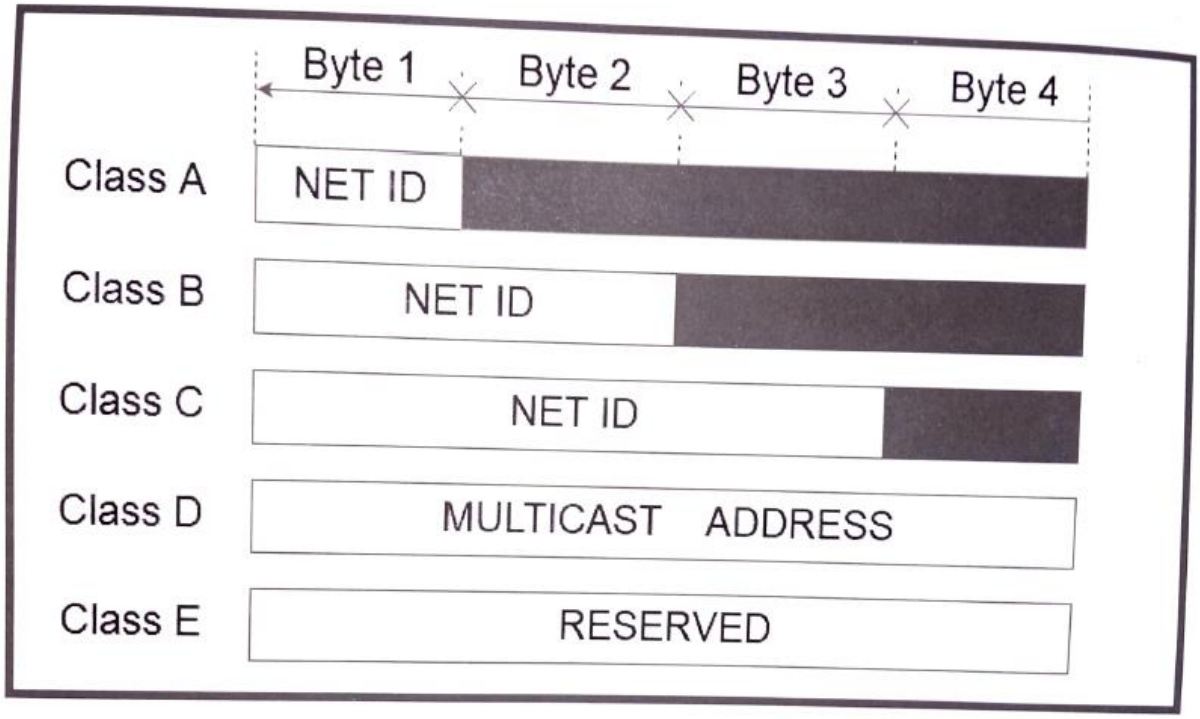
B) Introduction of Classful IP Addressing

When the Internet was in its initial stages (1981-1993), networks were simple and small. To assigning IP addresses and dividing the IPv4 address space, the classful IP addressing methods is used.

It was later replaced by CIDR (Classless Inter-Domain Routing), which uses prefixes instead of fixed classes. In classful IP addressing method, the 32 bits IP addresses were divided into five classes (A, B, C, D, E), which is easily identified by their first few bits.
Each of these classes has a valid range of IP addresses

Classes	Use
A, B and C	Used for unicast communication (large. medium, and small networks).
D	Reserved for multicasting
E	Experimental use

Each class of IP address uses a different number of bits for the network ID and host ID. which determines the total number of networks and hosts possible in that class.



Class A

In class A, the IP addresses are assigned to the networks that contain a large number ot hosts

	7 Bit	**24 Bit**
0	**Network**	**Host**

The higher-order bit of the first octet in class A is always set to 0. The remaining 7 hits in the first octet are used to determine network ID. The 24 bits of host ID are used to determine the host in any network. The default subnet mask for Class A is 255.X.X.X. Therefore, class A has a total of

2^{24}-2=16,777,214 host ID

Network ID	Host ID		
8-Bit	8-Bit	8-Bit	8-Bit
01111111	xxxxxxxx	xxxxxxxx	xxxxxxxx

IP addresses belonging to class A ranges from 0.0.0.0 to 127.255.255.255.

Class B

IP address belonging to class B is assigned to networks that range from medium-sized to large-sized networks

- The network ID is 16 bits long.
- The host ID is 16 bits long.

	14 Bit	**16 Bit**
10	**Network**	**Host**

The higher-order bits of the first octet of IP addresses of class B are always set to 10. The remaining 14 bits are used to determine the network ID. The 16 bits of host ID are used to determine the host in any network. The default subnet mask for class B is 255.255.x.x. Class B has a total of:

2^{14} = 16384 network address

2^{16}-2=65534 host address

Network ID	Host ID		
8-Bit	8-Bit	8-Bit	8-Bit
10111111	11111111	xxxxxxxx	xxxxxxxx

IP addresses belonging to class B ranges from 128.0.0.0 to 191.255.255.255.

Class C

IP addresses belonging to class Care assigned to small-sized networks.

- The network ID is 24 bits long
- The host ID is 8 bits long

	21 Bit	**8 Bit**
110	**Network**	**Host**

The higher-order bits of the first octet of IP addresses of class C is always set to 110. The remaining 21 bits are used to determine the network ID. The 8 bits of

host ID are used to determine the host in any network. The default subnet mask for class C is 255.255.255.x.

Class C has a total of:

2^{21} =2097152 network address

2^{8}-2= 254 host address

Network ID	Host ID		
8-Bit	8-Bit	8-Bit	8-Bit
11011111	11111111	11111111	xxxxxxxx

IP addresses belonging to class C range from 192.0.0.0 to 223.255.255.255.

Class D

IP address belonging to class D is reserved for multi-casting higher-order bits of the first octet of IP addresses belonging to class D is always set to 1110. The remaining bits are for the address that interested hosts recognize.

Clans D does not possess any subnet mask. IP addresses belonging class D range from 224.0.0.0 to 239.255.255.255

28 Bit

1110	**Host**

Class E

IP addresses belonging to class E are reserved for experimental and research purposes. addresses of claes E range from 240.0.0.0 to 255.255.255.255. This class doesn't have any subnet mask. The higher-order bits of the first octet of class E are always set to 1111

28 Bit

1111	**Host**

C) Classification Structure of classful addressing in terms of leading bits, Net ID bits, Host ID hits, No. of Network, address per network, start address and End address.

CLASS	LEADING BITS	NET ID BITS	HOST ID BITS	NO. OF NETWORKS	ADDRESSES PER NETWORK	START ADDRESS	END ADDRESS
CLASS A	0	8	24	2^7 (128)	2^{24} (16,777,216)	0.0.0.0	127.255.255.255
CLASS B	10	16	16	2^{14} (16,384)	2^{16} (65,536)	128.0.0.0	191.255.255.255
CLASS C	110	24	8	2^{21} (2,097,152)	2^8 (256)	192.0.0.0	223.255.255.255
CLASS D	1110	NOT DEFINED	NOT DEFINED	NOT DEFINED	NOT DEFINED	224.0.0.0	239.255.255.255
CLASS E	1111	NOT DEFINED	NOT DEFINED	NOT DEFINED	NOT DEFINED	240.0.0.0	255.255.255.255

Procedure:

1. Write the given IP addresses in dotted-decimal format.

2. Note the first octet (the number before the first dot).

3. Compare this octet with the class range table above.

4. Identify the class and write down the corresponding

- Network/Host structure
- Default subnet mask
- Purpose of the address class

5. Record all results in the observation table.

Observation Table:

Sr. No.	Given IP Address	First Octet	Binary Form (1st Octet)	IP Class	Default Mask	Network/Host Structure	Purpose
1	10.5.25.7						
2	192.168.1.100						
3	203.5.113.25						

Result: Thus, studied about Class of the Given IP Addresses

Assignment Questions:

1. What are the five classes of IPv4 addresses?
2. Find the class of your PC's IP address.
3. What are the start and end address for different class of IP address?
4. Find the class of following IP address:
   a. 10.5.25.7, b. 192.168.1.100 c. 203.5.113.25
5. Which bits determine the class of an IP address?
6. What is default subnet mask of class A, B and C?

Signature of Instructor

S.Y.B.C.A. (Science) Semester-IV (NEP-2024 Pattern)
Lab Experiment No.-7
Arduino based LED pattern generation

Aim: To study Arduino based LED pattern generation.

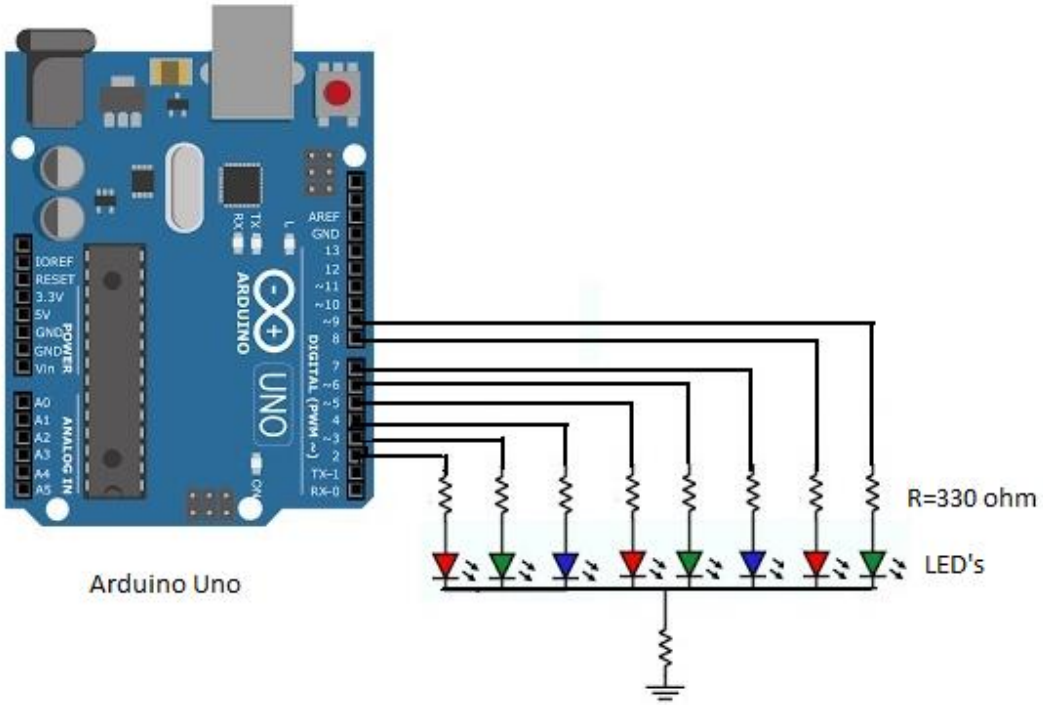
Objectives:

1. To understand digital I/O pin configuration in Arduino.
2. To interface multiple LEDs with Arduino Uno.
3. To write an embedded C program to generate different LED patterns.
4. To observe and analyze timing and sequence of LED blinking.

Apparatus/Software Required:

1. Arduino Uno Board
2. LEDs (8 Nos.)
3. 220 Q resistors (8 Nos.)
4. Circuit board/ Breadboard and connecting wires
5. USB cable for uploading code
6. Arduino IDE software

Circuit Diagram:



Algorithm:

1. Start
2. Initialize LED pins as OUTPUT
3. Define delay time for pattern speed
4. Turn ON LEDs in desired sequence (pattern)
5. Wait for specified delay
6. Turn OFF LEDs (or change pattern)
7. Repeat steps 4–6 to generate continuous pattern
8. Stop (or loop continuously)

Program: LED Interfacing to Arduino Uno

```
void setup() {
  for (int i = 2; i <= 9; i++) {
    pinMode(i, OUTPUT);
  }
}

void loop() {
  // All LEDs ON
  for (int i = 2; i <= 9; i++) digitalWrite(i, HIGH);
  delay(500);

  // All LEDs OFF
  for (int i = 2; i <= 9; i++) digitalWrite(i, LOW);
  delay(500);

  // Left to Right
  for (int i = 2; i <= 9; i++) {
    digitalWrite(i, HIGH);
    delay(200);
    digitalWrite(i, LOW);
  }

  // Right to Left
  for (int i = 9; i >= 2; i--) {
```

```
    digitalWrite(i, HIGH);
    delay(200);
    digitalWrite(i, LOW);
  }

  // Alternate Blink
  for (int i = 2; i <= 9; i += 2) digitalWrite(i, HIGH);
  for (int i = 3; i <= 9; i += 2) digitalWrite(i, LOW);
  delay(500);

  for (int i = 2; i <= 9; i += 2) digitalWrite(i, LOW);
  for (int i = 3; i <= 9; i += 2) digitalWrite(i, HIGH);
  delay(500);

  // Turn all off
  for (int i = 2; i <= 9; i++) digitalWrite(i, LOW);
}
```

Theory:

1. **Arduino Uno:** The Arduino Uno is a microcontroller board based on the ATmega328P. It has 14 digital 1/O pins, of which 6 can be used as PWM outputs. It operates at 5V DC and can be programmed via USB using the Arduino IDE.

Key Functions

pinMode(pin, mode)

- Purpose: Configures a pin as INPUT or OUTPUT
- Syntax: pinMode(pin, mode)
- Example: pinMode(13, OUTPUT)

digitalWrite(pin, value)

- Purpose: Writes HIGH (5V) or LOW (0V) to digital pin
- Syntax: digitalWrite(pin, value)
- Example: digitalWrite(13, HIGH)

analogWrite(pin, value)

- Purpose: Writes analog value (PWM) to pin (0-255)
- Syntax: analogWrite(pin, value)
- Example: analogWrite(9, 128) // 50% brightness

Common Arduino Pins for LEDs

Pin Type	Pin Numbers	Use Case
Digital Output	2-13	Simple on/off control
PWM Pins (-)	3, 5, 6, 9, 10, 11	Brightness control
Built-in LED	13 (or LED BUILTIN)	Testing without external components

2. LED (Light Emitting Diode): An LED emits light when current passes through it in the forward direction. It must be connected with a series resistor to limit current, typically 220 2.

3. Digital Output Control: Each digital pin can be set as OUTPUT and driven HIGH (SV) or LOW (OV) using:

```
pinMode ipin, OUTPUT);
digitalWrite(pin, HIGH); // Turn ON LED
digitalWrite(pin, LOW); // Turn OFF LED
```

4. LED Pattern Generation: Different LED patterns can be created by switching LEDs ON and OFF in specific time intervals using delay functions.

Result and conclusion: The Arduino Uno successfully generated multiple LED blinking patterns as programmed. The circuit worked as expected, confirming correct interfacing and timing control.

This experiment demonstrates the use of Arduino for digital output control and pattern generation using LEDs. It helps understand basic embedded programming. GPIO operation, and timing control using delays.

Signature of Instructor

S.Y.B.C.A. (Science) Semester-IV (NEP-2024 Pattern)
Lab Experiment No.-8
To study Bluetooth.

Aim: To study Bluetooth.

Objectives:

1. To understand the basic working principle of Bluetooth wireless communication.

2. To study the architecture and components of a Bluetooth system.

3. To understand the Bluetooth communication process, including pairing and data transmission.

4. To demonstrate basic Bluetooth-based wireless data communication between devices.

5. To identify applications of Bluetooth technology in short-range wireless communication systems.

Apparatus/Software Required:

1. Arduino UNO board

2. Bluetooth module (HC-05 or HC-06)

3. Jumper wires

4. Breadboard

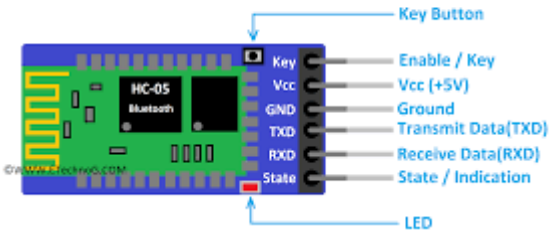
5. LED (for testing data transfer)

6. Android smartphone with Bluetooth terminal app

7. USB cable

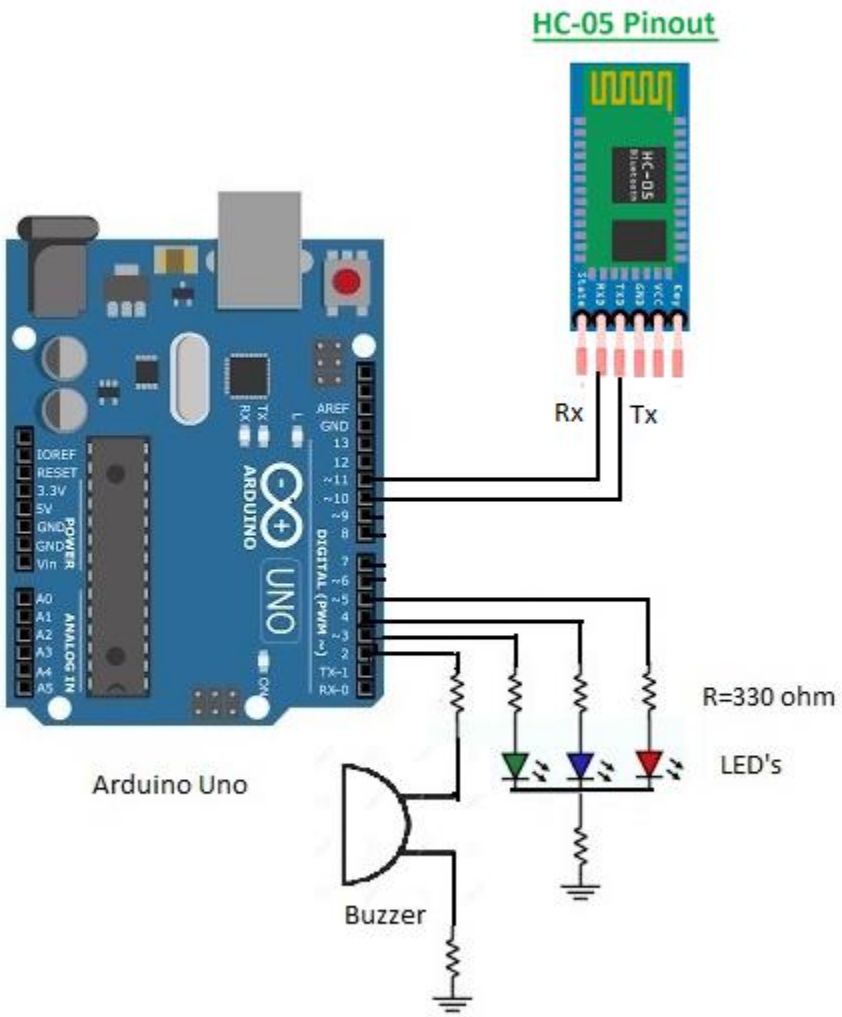
Bluetooth Module HC05 and its connections:

Figure.01: Bluetooth Module HC-05 Pinout Diagram



Frequency Band: 2.402 GHz to 2.480 GHz
Range: Typically 10 meters (can extend up to 100 meters in Class 1 devices).

Circuit Diagram:



Algorithm: Bluetooth Communication

1. Start
2. Initialize Bluetooth module
3. Enable Bluetooth on both devices
4. Search for nearby Bluetooth devices
5. Select target device
6. Pair devices using passkey
7. Establish connection
8. Send/Receive data
9. Check connection status
10. If connection lost → Reconnect
11. Stop

Program: Bluetooth Module Interfacing to Arduino Uno

```
#include <SoftwareSerial.h>
SoftwareSerial bluetooth(10, 11);  // TX, RX
char data;
#define BUZZER_PIN 2
#define RED_LED 3
#define YELLOW_LED 4
#define GREEN_LED 5

void setup() {

  pinMode(YELLOW_LED, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);

  Serial.begin(9600);
  bluetooth.begin(9600);

  digitalWrite(YELLOW_LED, LOW);
  digitalWrite(RED_LED, LOW);
  digitalWrite(GREEN_LED, LOW);
```

```
  digitalWrite(BUZZER_PIN, HIGH);   // OFF (Active LOW)

  Serial.println("Bluetooth Control Ready (Active LOW Buzzer)");
}

void loop() {

  if (bluetooth.available()) {

    data = bluetooth.read();

    Serial.print("Received: ");
    Serial.println(data);

    switch(data) {

      case 'Y':
        digitalWrite(YELLOW_LED, HIGH);
        Serial.println("Yellow LED ON");
        break;

      case 'y':
        digitalWrite(YELLOW_LED, LOW);
        Serial.println("Yellow LED OFF");
        break;

      case 'B':
        buzzerBeep();
        Serial.println("Buzzer Activated");
        break;

      case 'R':
        toggleRedLED();
        Serial.println("Red LED Toggled");
```

```
        break;

      case 'G':
        toggleGreenLED();
        Serial.println("Green LED Toggled");
        break;

      default:
        Serial.println("Invalid Command");
        break;
    }
  }
}

// ✅ Active LOW Buzzer Function
void buzzerBeep() {

  for(int i = 0; i < 3; i++) {
    digitalWrite(BUZZER_PIN, LOW);   // ON
    delay(200);
    digitalWrite(BUZZER_PIN, HIGH);  // OFF
    delay(150);
  }
}

void toggleRedLED() {
  digitalWrite(RED_LED, !digitalRead(RED_LED));
}

void toggleGreenLED() {
  digitalWrite(GREEN_LED, !digitalRead(GREEN_LED));
}
```

Theory: Bluetooth is a short-range wireless communication technology designed for exchanging data between fixed and mobile devices over short distances using UHF radio waves in the 2.4 GHz ISM band.

It follows the IEEE 802.15.1 standard and uses Frequency Hopping Spread Spectrum (FHSS) for secure communication. Bluetooth supports both point-to-point and point-to-multipoint connections through Piconets and Scattermets.

Bluetooth Architecture:

Master Device: Controls the communication and synchronization

Slave Device: Responds to commands from the master.

Piconet: One master and up to seven active slave devices.

Scatternet: Multiple interconnected piconets

Procedure:

1. Connect the HC-05 module to the Arduino as per the circuit diagram.
2. Upload the program to the Arduino board
3. Open a Bluetooth terminal app on your smartphone.
4. Pair your smartphone with the HC-05 module (default password: 1234 or 0000)
5. Send character 'T' to turn ON the LED and ' to turn it OFF.

Result: The Bluetooth module (HC-05) successfully established wireless serial communication with the Arduino. The LED responded correctly to commands from the smartphone app

Conclusion: Bluetooth communication allows simple and reliable short-range wireless data exchange between embedded systems and mobile devices, making it ideal for IoT and home automation applications.

Applications:

1. Wireless data transfer between devices
2 IoT communication (Arduino, sensors)
3. Mobile and PC peripherals (keyboards, mice, speakers)
4. Home automation and robotics

Signature of Instructor

S.Y.B.C.A. (Science) Semester-IV (NEP-2024 Pattern)
Lab Experiment No.-9
To study RFID

Aim: To study the RFID.

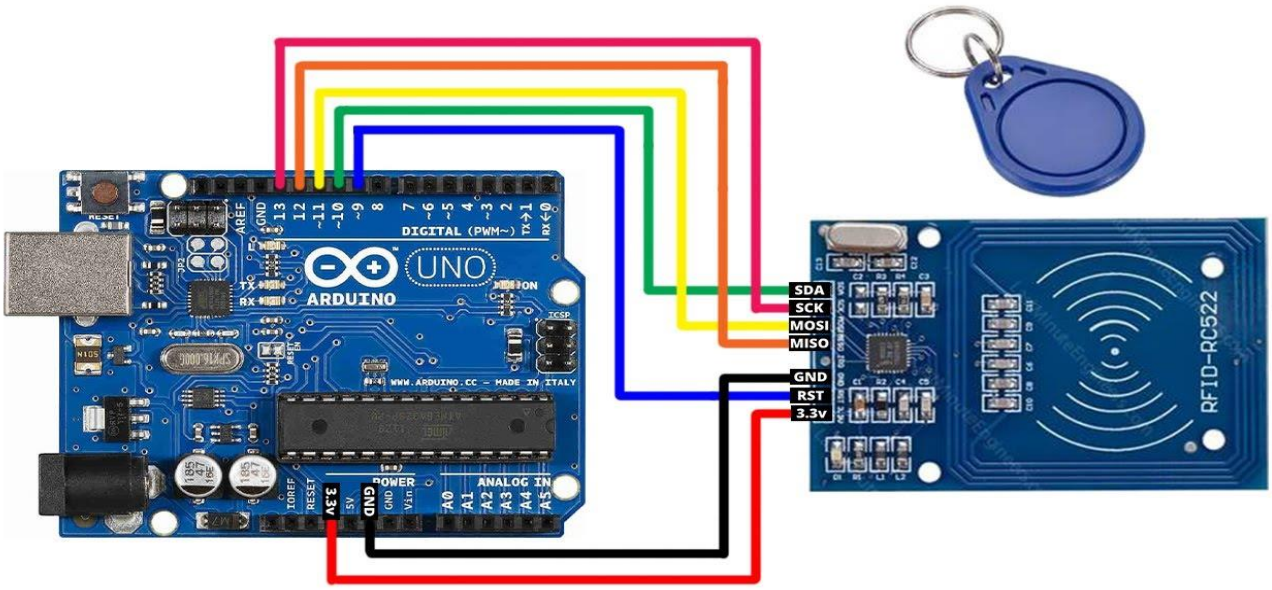
Objectives:

1. To understand the working principle of Radio Frequency Identification (RFID) technology.
2. To study the architecture and components of an RFID system
3. To identify different types of RFID tags and readers.
4. To understand the communication process between RFID tag and reader.
5. To demonstrate a basic RFID system using Arduino and an RFID reader module.

Apparatus/Software Required:

1. Arduino UNO board
2. RFID Reader Module (RC522)
3. RFID Tags/Cards
4. Jumper wires
5. Breadboard
6. USB cable
7. Arduino IDE software

Circuit Diagram:



Algorithm: RFID System

1. Start the system
2. Initialize RFID reader module
3. Wait for RFID tag near the reader
4. Read data from RFID tag
5. Compare tag ID with stored database
6. IF tag is valid
   - Grant access / display authorized message
7. ELSE
   - Deny access / show alert
8. Repeat the process continuously
9. Stop

Program: RFID Module Interfacing to Arduino Uno

```
/*
  Typical pin layout used:
  -----------------------------------------------------------------------------------------
                          MFRC522      Arduino       Arduino    Arduino    Arduino          Arduino
                          Reader/PCD   Uno/101       Mega       Nano v3    Leonardo/Micro   Pro Micro
  Signal                  Pin          Pin           Pin        Pin        Pin              Pin
  -----------------------------------------------------------------------------------------
  RST/Reset               RST          9             5          D9         RESET/ICSP-5     RST
  SPI SS                  SDA(SS)      10            53         D10        10               10
  SPI MOSI                MOSI         11 / ICSP-4   51         D11        ICSP-4           16
  SPI MISO                MISO         12 / ICSP-1   50         D12        ICSP-1           14
  SPI SCK                 SCK          13 / ICSP-3   52         D13        ICSP-3           15
*/
```

```
#include <SPI.h>
#include <MFRC522.h>

#define SS_PIN 10
#define RST_PIN 9
#define OK_LED 3 // connection
#define ERROR_BUZ 4 //connection

MFRC522 rfid(SS_PIN, RST_PIN); // Instance of the class
MFRC522::MIFARE_Key key;

// Unique ID of RFID Tag which you want to give access
int My_RFID_Tag[4] = {0x69, 0xD2, 0xCB, 0x06};
```

```
// 69D2CB06 - RFID card


boolean My_Card = false; // variable to hold your Access_card
byte nuidPICC[4]; // array that will store new NUID
String lastSent;

void setup() {
  pinMode(OK_LED, OUTPUT);
  pinMode(ERROR_BUZ, OUTPUT);

  digitalWrite(OK_LED, LOW);
  digitalWrite(ERROR_BUZ, HIGH);

  Serial.begin(9600);
  SPI.begin(); // Init SPI bus
  rfid.PCD_Init(); // Init MFRC522
}

void loop() {
  // First assume detected card is NOT My_Card
  My_Card = false;

  if (!rfid.PICC_IsNewCardPresent())
    return;

  if (!rfid.PICC_ReadCardSerial())
    return;

  MFRC522::PICC_Type piccType =
rfid.PICC_GetType(rfid.uid.sak);

  if (piccType != MFRC522::PICC_TYPE_MIFARE_MINI &&
      piccType != MFRC522::PICC_TYPE_MIFARE_1K &&
```

```
      piccType != MFRC522::PICC_TYPE_MIFARE_4K) {
    Serial.println(F("Your tag is not of type MIFARE
Classic.")); 
    return;
  }

  printHex(rfid.uid.uidByte, rfid.uid.size);
// display card number on serial monitor
  Serial.println();

  // Compare this RFID Tag Unique ID with your My_RFID_Tag's Unique ID
  My_Card = true; // Assume it's valid until proven otherwise
  for (int i = 0; i < 4; i++) {
    if (My_RFID_Tag[i] != rfid.uid.uidByte[i]) {
      My_Card = false;
      break;
    }
  }

  // If RFID Tag is My_Card then grant access
  if (My_Card) {
    Serial.println("\nWelcome To Class Room");
    Serial.println("Access Granted");

    digitalWrite(ERROR_BUZ,HIGH );
    digitalWrite(OK_LED, HIGH);
    delay(100);
    digitalWrite(OK_LED, LOW);
  } else {
    Serial.println("\nDon't enter in the Class Room");
    Serial.println("Card NOT FOUND!");
    Serial.println("Access Denied");

    digitalWrite(OK_LED, LOW);
```

```
    digitalWrite(ERROR_BUZ, LOW);
    delay(150);
    digitalWrite(ERROR_BUZ, HIGH);
  }

  // Halt PICC and stop encryption
  rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();
}

/**Helper routine to dump a byte array as hex values to Serial*/
void printHex(byte *buffer, byte bufferSize) {
  lastSent = "";
  for (byte i = 0; i < bufferSize; i++) {
    byte n1 = (buffer[i] >> 4) & 0x0F;
    byte n2 = buffer[i] & 0x0F;
    lastSent += n1 < 10 ? '0' + n1 : 'A' + (n1 - 10);
    lastSent += n2 < 10 ? '0' + n2 : 'A' + (n2 - 10);

    Serial.print(buffer[i] < 0x10 ? "0" : "");
    Serial.print(buffer[i], HEX);
  }
}
```

Theory: Radio Frequency Identification (RFID) is a wireless communication technology that uses radio waves to identify and track objects automatically. Each RFID system consists of a reader and one or more tags.

The RFID system is comprised of two components: the RFID reader and the tags. They are also called PCD (Proximity Coupling Device) and PICC (Proximity Integrated Circuit Card). The RFID reader consists of an antenna to emit high-frequency EM waves and a reader/writer. MFRC522 from NXP is an example of such an integrated circuit. Since we are using high-frequency waves in the megahertz range, the size of the antenna can be small.

The RFID tag can be either passive or active. Active tags are powered by batteries, while the passive RFID tags are powered by energy from the reader's interrogating EM waves. The tags are available in different forms or shapes, like cards, tags, key fobs, or stickers, Whatever the shape, the RFID tag will consist of an antenna and the RFID chip, which will store all the data. When triggered by an electromagnetic interrogation pulse from a nearby RFID reader, the tag will transmit data back to the reader. The reader will then analyze this data to identify the tag. Unlike a barcode or a QR code, the tag does not need to be within the reader's line of sight. This makes it easier to process and can be used for tracking objects in a closed space

RC522 Module Overview:

The RC522 RFID module is based on the popular MFRC522 RFID reader chip from NXP. These modules are cheap and available from most online stores. MFRC522 is a highly integrated RFID reader/writer IC for contactless communication at 13.56 MHz. The MFRC522 reader supports ISO/IEC 14443 A/MIFARE and NTAG. The operating voltage of the RC522 module is 2.5V 3.3V. Even though the maximum supply voltage is 3.3V, the communication pins are 5V tolerant. So, we can connect the module directly to an Arduino without any level shifters.

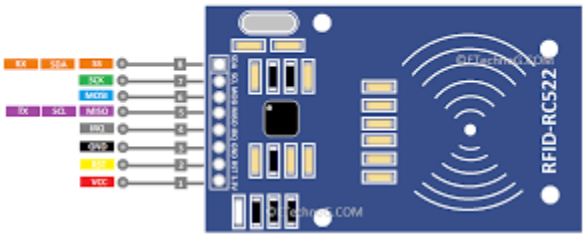
The MFRC522 supports three different communication protocols:

- SPI with Speed up to 10Mbit/s
- 12C interface with speed up to 400kBd in Fast mode and up to 3400kBd in High-Speed mode
- RS232 Serial UART with speed up to 1228.8kBd

The RC522 RFID module is usually supplied with a card and a key fob, both of which are MIFARE Classic tags with IKB of storage. We can not only read these tags but can also program them with the RC522 module. Here is the picture showing the tags along with the RC522 module

RC522 Pinout: The RC522 module has a total of 8 pins. This module supports various communication protocols, and each pin has a different function for each communication protocol.

Figure.01: RFID RC522 Pinout Diagram



Basic Working Principle:

1. The RFID reader generates an electromagnetic field.
2. When an RFID tag comes within range, it absorbs energy from this field.
3. The tag transmits its unique identification number (UID) back to the reader.
4. The reader then sends this data to a microcontroller or computer for processing.

Types of RFID Systems:

Type	Frequency Range	Read Range	Applications
Low Frequency (LF)	125-134 kHz	Up to 10 cm	Access control, animal tagging
High Frequency (HF)	13.56 MHz	Up to 1 meter	Smart cards, library systems
Ultra High Frequency (UHF)	860-960 MHz	Up to 12 meters	Inventory management, logistics

Components of an RFID System:

1. RFID Tag: Contains a microchip and antenna. Can be passive, active, or semi-passive.
2. RFID Reader: Emits radio waves and receives tag data.
3. Antenna: Enables communication between tag and reader.
4. Host System: Processes and stores tag data.

RFID Reader (RC522) Pin Configuration:

Pin	Description
VCC	+3.3V Power Supply
GND	Ground
RST	Reset Pin
IRQ	Interrupt Pin (optional)

MISO	Master In Slave Out
MOSI	Master Out Slave In
SCK	Serial Clock
SDA	Slave Select

Procedure:

1. Connect the RFID Reader (RC522) to the Arduino UNO as per the circuit diagram.
2. Upload the given code to the Arduino using the Arduino IDE.
3. Open the Serial Monitor (9600 baud rate).
4. Place an RFID card/tag near the reader.
5. Observe the Unique ID (UID) displayed on the Serial Monitor.

Result: The RFID system successfully read and displayed the unique identification number (UID) of each RFID tag when placed within range of the RC522 reader module.

Conclusion: RFID technology enables contactless identification of objects using radio frequency waves. It is widely used in security systems, asset tracking, inventory control, and automated access systems.

Applications:

1. Library management systems
2. Attendance monitoring
3. Access control and authentication
4. Supply chain and inventory tracking
5. Smart retail systems

Signature of Instructor

S.Y.B.C.A. (Science) Semester-IV (NEP-2024 Pattern)
Lab Experiment No.-11
Wireless sensor node used for Street Light Control

Aim: Wireless sensor node used for street light control.

Objectives:

1. To understand the concept of a wireless sensor node used in smart street lighting systems.

2. To study the working principle of LDR (Light Dependent Resistor) for sensing ambient light intensity.

3. To interface an LDR sensor with Arduino for light-level detection.

4. To design and implement automatic street light control logic based on ambient light conditions.

5. To establish wireless communication using ZigBee or RF modules for sensor data transmission.

Apparatus/Software Required:

1. Arduino UNO (2 units-Transmitter & Receiver nodes)

2. ZigBee (XBee) or RF modules (pair)

3. LDR (Light Dependent Resistor)

4. 10 k resistor

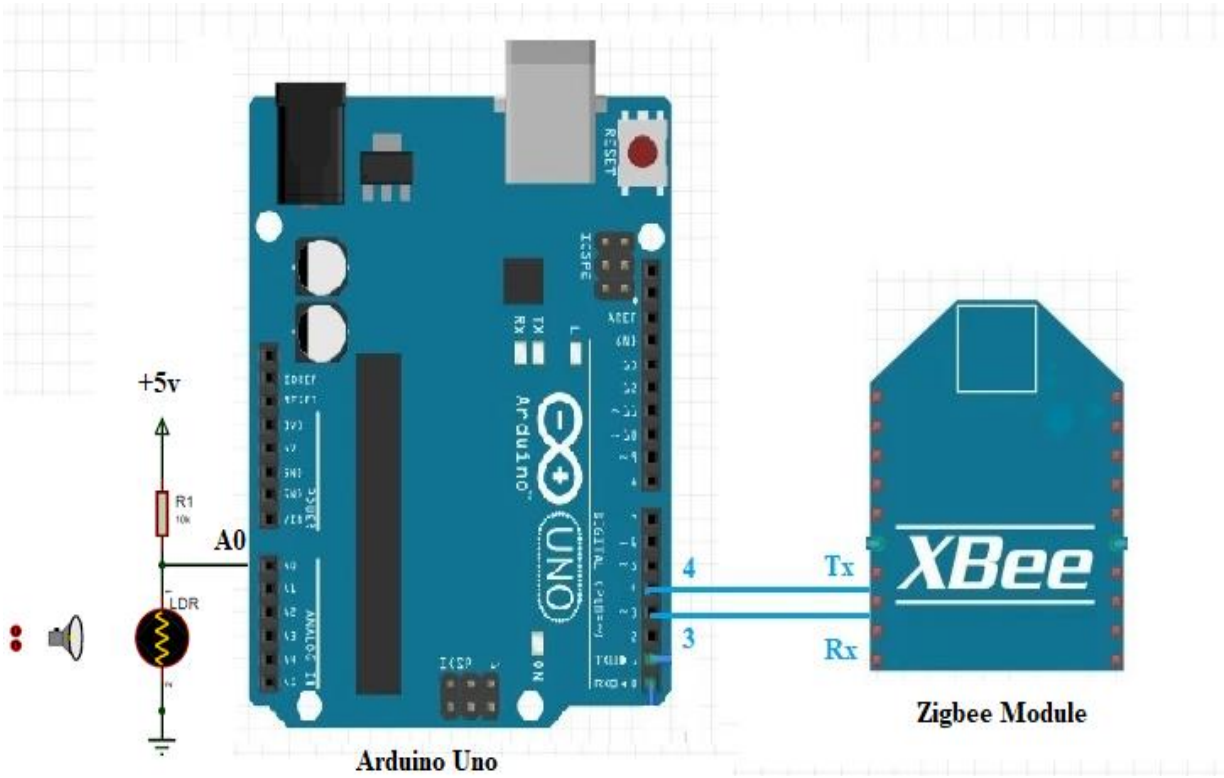
5. LED(s) to represent street lights

6. Breadboard and jumper wires

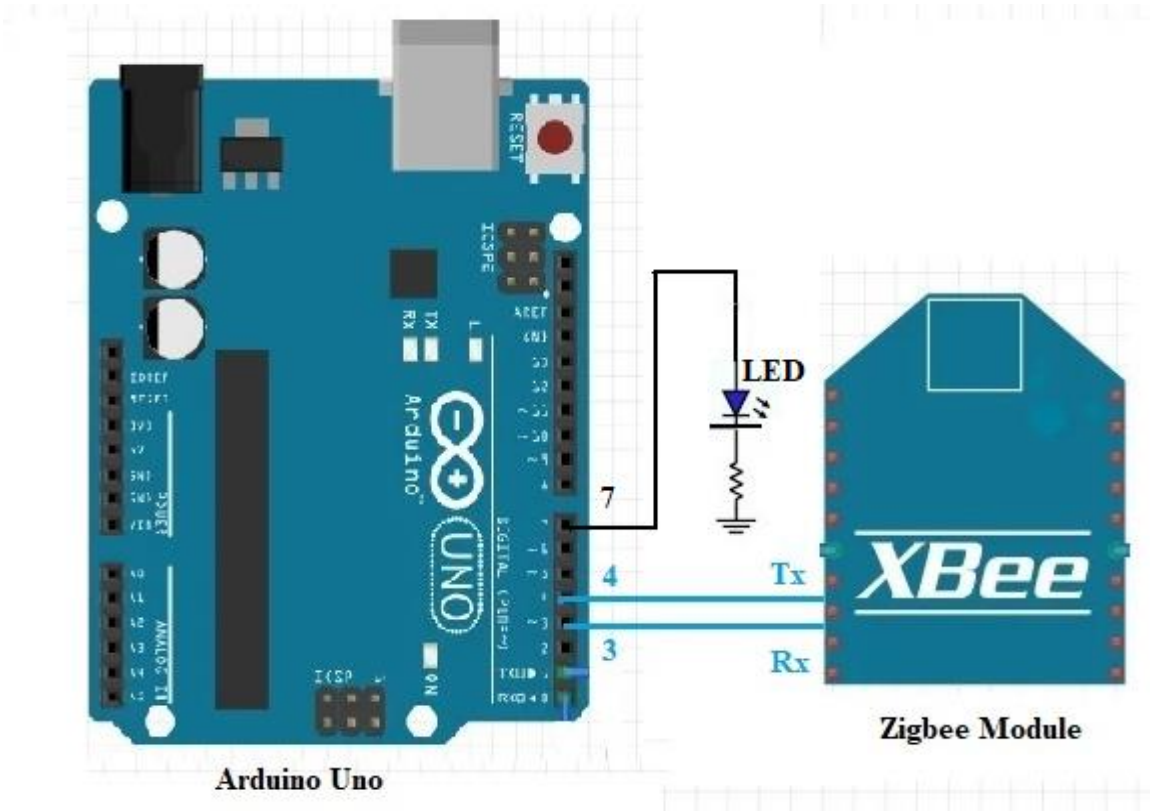
7. USB cable

8. Arduino IDE software

Circuit Diagram: Zigbee Transmitter Node:



Circuit Diagram: Zigbee Receiver Node



Algorithm: Street Light Control using Wireless Sensor Node

1. Start the system
2. Initialize microcontroller, LDR (light sensor), motion sensor, and wireless module
3. Read light intensity from LDR
4. IF daylight (high light intensity):
   - Turn OFF street lights
5. ELSE (night condition):
   - Turn ON street lights
6. Check motion sensor
7. IF motion detected:
   - Increase light brightness
8. ELSE:
   - Keep lights at dim level (to save energy)
9. Send status data to control station via wireless module
10. Repeat steps continuously

Program: street light control using Arduino Uno (Tx)

```
// TRANSMITTER NODE - LDR SENSOR (Using SoftwareSerial)

#include <SoftwareSerial.h>

//XBee TX → Arduino pin 3
//XBee RX → Arduino pin 4
SoftwareSerial xbee(3, 4);   // RX, TX

int LDR = A0;
int value = 0;

void setup() {
  Serial.begin(9600);   // PC Monitor
  xbee.begin(9600);     // XBee communication
}

void loop() {

  value = analogRead(LDR);
```

```
  // Send value via XBee
  xbee.println(value);

  // Also print on Serial Monitor
  Serial.print("Sent LDR Value: ");
  Serial.println(value);

  delay(1000);
}
```

Program: street light control using Arduino Uno (Rx)

```
// RECEIVER NODE - STREET LIGHT CONTROL
// Using SoftwareSerial

#include <SoftwareSerial.h>

//XBee TX → Arduino pin 3
//XBee RX → Arduino pin 4
SoftwareSerial xbee(3, 4);   // RX, TX

int LED = 7;
int sensorValue = 0;

void setup() {
  Serial.begin(9600);   // PC Monitor
  xbee.begin(9600);     // XBee communication
  pinMode(LED, OUTPUT);
}

void loop() {

  if (xbee.available()) {
```

```
    sensorValue = xbee.parseInt();

    Serial.print("Received LDR Value: ");
    Serial.println(sensorValue);

    if (sensorValue < 500) {      // Night condition
      digitalWrite(LED, HIGH);    // Street Light ON
      Serial.println("Street Light ON");
    }
    else {                        // Day condition
      digitalWrite(LED, LOW);     // Street Light OFF
      Serial.println("Street Light OFF");
    }
  }
}
```

Theory: A Wireless Sewor Node (WSN) is a small electronic device equipped with sensors, a microcontroller, and a communication module

It senses environmental conditions, processes data, and transmits it wirelessly to a base station or another node.

In a street light control system, each node monitors ambient light intensity using an LDR sensor and controls street lights automatically.
The data is also sent wirelessly to a receiver or control center for monitoring

Working Principle:

1. The LDR changes its resistance based on the surrounding light.

2. When the light intensity decreases (night), the Arduino turas ON the LED street light).

3. When light intensity increases (day), the LED is turned OFF

4. The transmitter node sends the LDR status wirelessly to the receiver node, which then operates the street light indicator.

Procedure:

1. Connect the LDR and Zigbee modules to the transmitter Arduino as per the circuit diagram

2. Connect the LED and ZigBee module to the receiver Arduino.

3. Upload the respective transmitter and receiver programs.

4. Open the Serial Monitor on the receiver side to observe incoming sensor values.

5. Observe that the LED turns ON when light intensity is low (night condition) and turns OFF when it is high (day condition).

Observation Table:

Sr. No.	Light Intensity (LDR Value)	Wireless Transmission	LED Status	Remark
1	850	Sent	OFF	Daylight
2	420	Sent	ON	Night time
3	700	Sent	OFF	Cloudy day
4	300	Sent	ON	Dark

Result: A wireless sensor node-based street light control system was successfully implemented. The LED (street light) automatically switched ON in darkness and OFF in brightness. demonstrating wireless control using ZigBee modules.

Conclusion: This experiment demonstrates how wireless sensor nodes can be used for smart street lighting
It provides an energy-efficient, automated, and wirelessly monitored lighting system an essential part of IoT-based Smart City applications.

Applications:

- Smart city street lighting systems
- Energy-efficient lighting control
- Wireless environmental monitoring
- Remote automation in public infrastructure

Signature of Instructor

S.Y.B.C.A. (Science) Semester-IV (NEP-2024 Pattern)
Lab Experiment No.-13
Wireless sensor node used for security system.

Aim: Wireless sensor node used for security system.

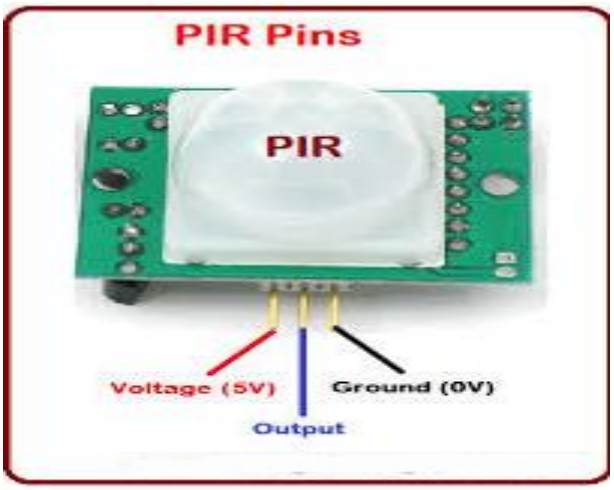
Objectives:

1. To understand the role of wireless sensor nodes in security applications.
2. To interface a PIR (Passive Infrared) sensor with a microcontroller.
3. To transmit an intrusion alert wirelessly using RF/ZigBee/Bluetooth modules.
4. To demonstrate an IoT-based prototype for security monitoring.

Apparatus/Software Required:

Sr. No.	Component	Description
1	Microcontroller	Arduino Uno
2	PIR Sensor	HC-SR501 Motion Detector
3	Wireless Module	XBee
4	LED	Indicator for intrusion
5	Buzzer	Audio alert
6	Power Supply	5V DC
7	Connecting Wires	Male-to-Male/Female-to-Female jumpers

Module:

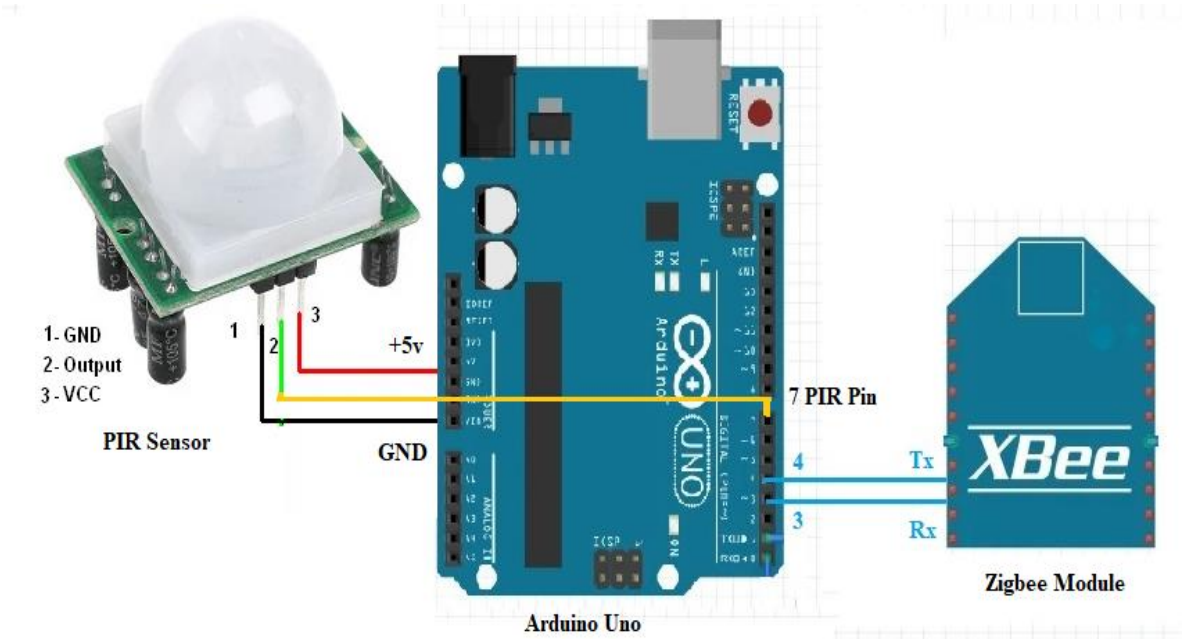


Theory: A wireless sensor node used for a security system is a small, low-power electronic device that detects security-related events (like motion, intrusion, fire, or tampering) and wirelessly sends data to a control unit or monitoring system.

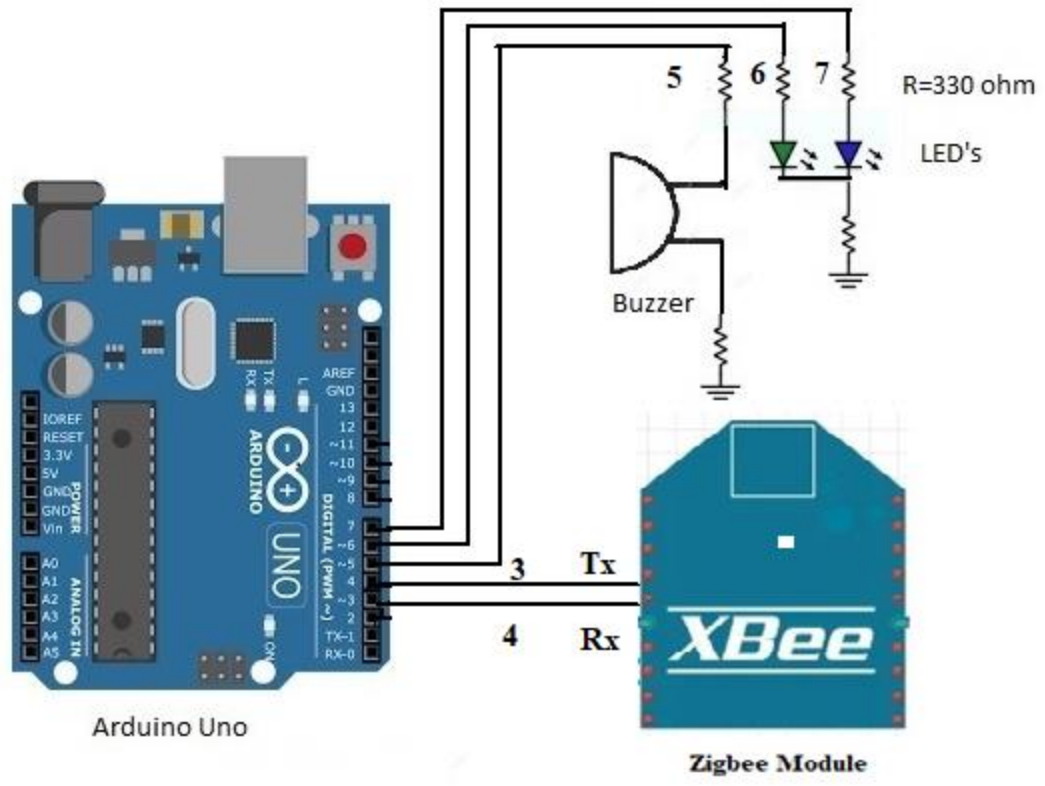
To detect security related event PIR sensors can be used. We can also me gas or smoke sensor to detect the fire also. The microcontroller unit, such as an Arduino boand, controls the operation of the node by processing sensor data and making decisions.

Wireless communication modules, such as Bluetooth, enable remote communication. Buzzer, siren, or LED are used for local alerts

Circuit Diagram: Transmitter



Circuit Diagram: Receiver



Algorithm:

1. Start the system
2. Initialize sensors and wireless module
3. Read sensor data continuously
4. IF intrusion/motion detected
   → Activate alarm
   → Send alert to base station
5. ELSE continue monitoring
6. Enter sleep mode when idle (power saving)
7. Repeat steps continuously

Program: Wireless security system using Arduino Uno (Tx)

```
// TRANSMITTER NODE - PIR MOTION SENSOR
// Using SoftwareSerial for XBee

#include <SoftwareSerial.h>

//XBee TX → Arduino pin 3
//XBee RX → Arduino pin 4
SoftwareSerial xbee(3, 4);   // RX, TX

int pirPin = 7;
int lastState = LOW;

void setup() {
  pinMode(pirPin, INPUT);

  Serial.begin(9600);   // PC Monitor
  xbee.begin(9600);     // XBee communication

  Serial.println("PIR Transmitter Started");
  delay(2000);  // PIR stabilization
}

void loop() {

  int sensorValue = digitalRead(pirPin);

  // Send only when state changes (prevents spamming)
  if (sensorValue != lastState) {

    if (sensorValue == HIGH) {
      xbee.println("INTRUDER_ALERT");
      Serial.println("Motion Detected! Alert Sent.");
    }
```

```
    else {
      xbee.println("AREA_CLEAR");
      Serial.println("Area Clear.");
    }

    lastState = sensorValue;
  }

  delay(200);
}
```

Program: Wireless security system using Arduino Uno (Rx)

```
// RECEIVER NODE - SECURITY ALARM SYSTEM
// Using SoftwareSerial for XBee

#include <SoftwareSerial.h>

//XBee TX → Arduino pin 3
//XBee RX → Arduino pin 4
SoftwareSerial xbee(3, 4);   // RX, TX

int redLED = 7;
int greenLED = 6;
int buzzer = 5;      // Active LOW buzzer

String receivedMessage = "";

void setup() {
  Serial.begin(9600);   // PC Monitor
  xbee.begin(9600);     // XBee

  pinMode(redLED, OUTPUT);
  pinMode(buzzer, OUTPUT);
```

```
  pinMode(greenLED, OUTPUT);

  // Initial Safe State
  digitalWrite(redLED, LOW);
  digitalWrite(greenLED, HIGH);
  digitalWrite(buzzer, HIGH);   // OFF (Active LOW)

  Serial.println("Receiver Ready");
}

void loop() {

  if (xbee.available()) {

    receivedMessage = xbee.readStringUntil('\n');
    receivedMessage.trim();

    Serial.print("Received: ");
    Serial.println(receivedMessage);

    if (receivedMessage == "INTRUDER_ALERT") {
      activateAlarm();
    }
    else if (receivedMessage == "AREA_CLEAR") {
      deactivateAlarm();
    }
  }
}

void activateAlarm() {
  digitalWrite(redLED, HIGH);
  digitalWrite(greenLED, LOW);
  digitalWrite(buzzer, LOW);   // ON (Active LOW)
}
```

```
void deactivateAlarm() {
  digitalWrite(redLED, LOW);
  digitalWrite(greenLED, HIGH);
  digitalWrite(buzzer, HIGH);  // OFF
}
```

Procedures

1. Connect the PIR semor, LED, buzzer, and wireless module to the Arduino as per the circuit diagram.
2. Write and upload the Arduino code to detect motion and transmit an alert signal.
3. When motion is detected, the LED and buzzer will activate.
4. The wireless module will transmit the message "Intruder Detected to the receiver node or monitoring system.
5. Observe the output on the serial monitor or connected receiver device
6. Record observations for different test cases (motion/no motion).

Observation Table:

Sr. No.	Sensor Status	LED/Buzzer	Wireless Message
1	Motion Detected	ON	"Intruder Detected!"
2	No Motion	OFF	----

Result: The wireless sensor node successfully detected motion using the PIR sensor and transmitted, an intrusion alert wirelessly, confirming the feasibility of a sensor-based security system.

Conclusion: This experiment demonstrates how a wireless sensor node can monitor an environment for motion and transmit real-time alerts. Such systems form the basis of smart home security and IoT surveillance applications.

Questions:

1. What is a wireless sensor node?
2. How does a PIR sensor detect motion?
3. What are the advantages of using ZigBee or Bluetooth in security systems?
4. How can this system be extended for multiple rooms?
5. What are the possible limitations of this system?

Signature of Instructor